

Linked list: add a node to an ordered list

Advanced: Practice 2

Eli uses a system that keeps a list of the different birds he sees in his garden. The list of birds is stored within the system as a linked list:

- Each node has a distinct memory location, which is shown in **Figure 1** as the number above the node.
- Each node contains data (in this case, the name of the bird) and a pointer to the memory location of the next node.
- The node labelled **head** indicates the first element in the linked list (the **head** of the list).
- For the last element of the list, the next node pointer always points to a **null** value to mark the end of the list.
- The list is **ordered alphabetically (A to Z)**. When a new node is added, it is added **into the correct position** within the list.

The current state of the linked list is shown in **Figure 1** below:

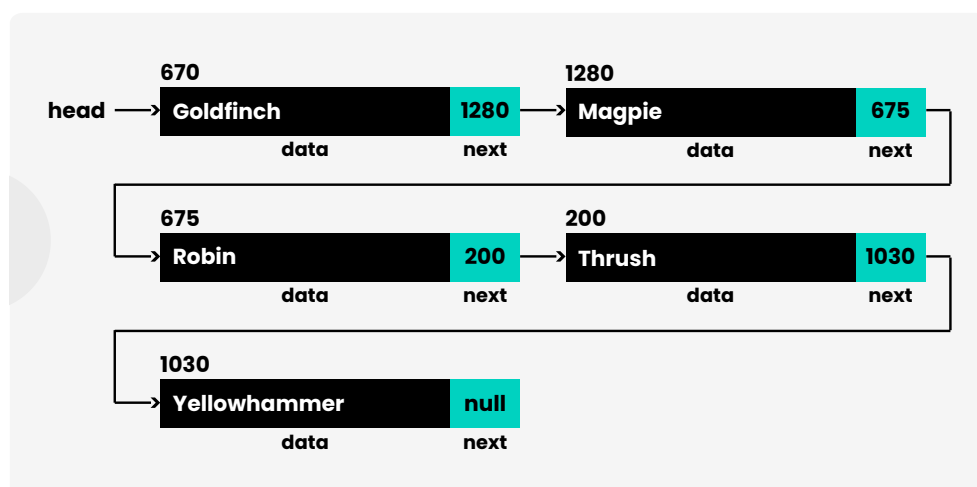


Figure 1: The current state of the linked list

Eli spots a pigeon in the garden and records it in the system.

A new node is created at memory location 950 to store the details of the bird. In this new node, the value of **data** will be set to "Pigeon". What value will **next** be set to?

Quiz:

STEM SMART Computer Science Week 14

Dictionary: definition 2

Core: Practice 1  

In a dictionary, items are accessed by whereas items stored in an array are accessed by . A dictionary is sometimes called an array.

Arrays are useful when you want to over all the stored values because they are stored in main memory. Dictionaries are a better choice if you want to access discrete values because you retrieve them by rather than by .

Items:

contiguously

iterate

ordered

index

key

associative

randomly

value

Quiz:

STEM SMART Computer Science Week 14

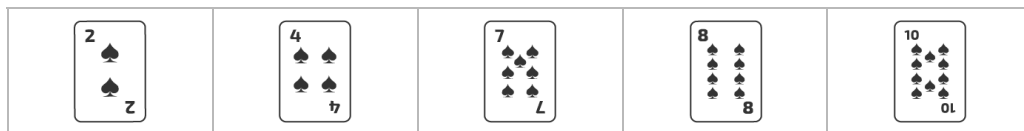
All teaching materials on this site are available under a CC BY-NC-SA 4.0 license, except where otherwise stated.



Binary or linear search? 1

Core: Challenge 1 

You have been provided with some cards that are in the following order:



Typically a binary search uses the formula $\text{midpoint} = (\text{first} + \text{last}) \text{ DIV } 2$ to calculate the midpoint position. Remember that integer or floor division (DIV) is when any remainder of a division is discarded.



For example, $11 \div 5 = 2$ remainder 1 and so 11 DIV 5 will return 2.

Part A

You need to find out if the number 4 is in the list of cards. Which searching algorithm would require the **fewest** number of comparisons to find the data?

- ☐ Linear
- ☐ Binary
- ☐ Both would be the same

Part B

You now need to find out if the number 9 is in the list of cards. Which searching algorithm would require the **fewest** number of comparisons to find the data?

- ☐ Binary
 - ☐ Linear
 - ☐ Both would be the same
-
-

Quiz:

STEM SMART Computer Science Week 14

All teaching materials on this site are available under a CC BY-NC-SA 4.0 license, except where otherwise stated.



Binary search: max comparisons 5

Advanced: Practice 1  

You are trying to find a movie in the media library stored on your computer, and decide to use a binary search algorithm to do it.

Your media collection has **100** titles in it. What is the **maximum number of comparisons** that could be made on your media collection while attempting to find a movie?

Here is a pseudocode version of the binary search algorithm:

Pseudocode

```
1 FUNCTION binary_search(data_set, item_sought)
2   index = -1
3   found = False
4   first = 0
5   last = LEN(data_set) - 1
6   WHILE first <= last and found == False
7     midpoint = (first + last) DIV 2
8     IF data_set[midpoint] == item_sought THEN
9       index = midpoint
10      found = True
11    ELSE
12      IF data_set[midpoint] < item_sought THEN
13        last = midpoint - 1
14      ELSE
15        first = midpoint + 1
16      ENDIF
17    ENDIF
18  ENDWHILE
19  RETURN index
20 ENDFUNCTION
```

Quiz:

[STEM SMART Computer Science Week 14](#)

All teaching materials on this site are available under a CC BY-NC-SA 4.0 license, except where otherwise stated.



Bubble sort: efficiency 1

Core: Challenge 1 

A bubble sort algorithm is written in pseudocode below. Study the pseudocode and then select **two** changes that could be made to improve the efficiency of the algorithm.

Pseudocode

```
1  PROCEDURE bubble_sort(items)
2      num_items = LEN(items)
3      FOR pass_num = 1 TO num_items - 1
4          FOR index = 0 TO num_items - 2
5              IF (items[index] > items[index + 1]) THEN
6                  temp = items[index]
7                  items[index] = items[index + 1]
8                  items[index + 1] = temp
9              ENDIF
10         NEXT index
11     NEXT pass_num
12 ENDPROCEDURE
```

- ☐ The variable **temp** is not needed when swapping items in this way and could be removed
- ☐ The outer for loop could be changed so that the number of swaps made is reduced by 1 after each pass
- ☐ The outer for loop could be changed to a while loop that stops once no swaps are made during a single pass
- ☐ The inner for loop could be changed so that the number of repetitions is reduced by 1 after each pass
- ☐ The inner for loop could be changed to a while loop that only swaps items if they are out of order

Quiz:

STEM SMART Computer Science Week 14

Insertion: trace 1

Advanced: Practice 2  

Pam is a teacher. She wants to sort her students test scores so that the highest score appears at the start of the list.

She uses an **insertion sort** to sort the data.

Drag and drop the lines into the correct order to show the state of the list of scores as the data is progressively sorted (i.e. as each item is correctly positioned).

The initial order of the list is: 43, 74, 64, 68, 49, 70

Available items

74, 43, 64, 68, 49, 70

74, 68, 64, 49, 43, 70

74, 70, 68, 64, 49, 43

74, 68, 64, 43, 49, 70

74, 64, 43, 68, 49, 70

Quiz:

STEM SMART Computer Science Week 14

All teaching materials on this site are available under a CC BY-NC-SA 4.0 license, except where otherwise stated.



Merge sort: trace 3

Core: Practice 1

You have been given some playing cards that have not been sorted:

Original order							
							

Part A

Drag and drop the cards provided to show the order they would be in at the **end** of each **step** of a **merge sort**.

For this part of the question, you just need to show the **splitting** part of the algorithm.

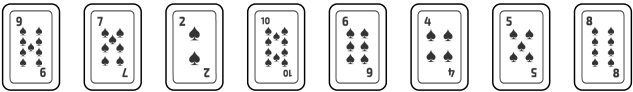
Splitting

Step 1	group 1				group 2			

Step 2	group 1		group 2		group 3		group 4	

Step 3	group 1	group 2	group 3	group 4	group 5	group 6	group 7	group 8

Items:



Part B

You have been given some playing cards that have not been sorted:

Original order							
							

Drag and drop the cards provided to show the order they would be in at the **end** of each **step** of a **merge sort**.

For this part of the question, you just need to show the **merging** part of the algorithm. You will need to use your answer from part 1 of the question to help you.

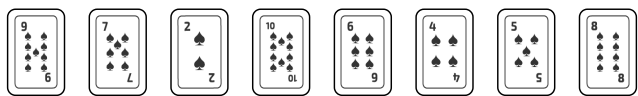
Merging

Step 4	group 1		group 2		group 3		group 4	
								

Step 5	group 1				group 2			
								

Step 6	Sorted cards							
								

Items:



Recursion: advantages

Advanced: Challenge 2 

A recursive algorithm can always be written to use iteration (instead of recursion). The following statements show possible advantages of using recursion. Which **two** are correct?

- ☐ Problems that are naturally expressed by recursion are easier to code
 - ☐ Recursive algorithms are easier to trace
 - ☐ Recursive algorithms use less memory
 - ☐ There are usually fewer lines of code
-
-
-

Quiz:

STEM SMART Computer Science Week 14

All teaching materials on this site are available under a CC BY-NC-SA 4.0 license, except where otherwise stated.



Recursion: trace table

Advanced: Challenge 2

A recursive subroutine has been written below. A trace table has been partially filled in, showing the state of the variables and condition, alongside the number of calls to the `make_num` subroutine and the return value.

Write the correct value in the correct location in the trace table:

Pseudocode

```
1 FUNCTION make_num(x, y)
2     IF x == 0 THEN
3         RETURN y
4     ELSE
5         RETURN make_num(x-2, y-1)
6     ENDIF
7 ENDFUNCTION
8
9 make_num(6, 7)
```

Call #	x	y	x equals 0?	Return
1	6	7	False	make_num(4, 6)
2				
3				
4				

Quiz:
STEM SMART Computer Science Week 14

Recursion: trace the code 1

Advanced: Challenge 2 

A recursive subroutine has been written as follows:

Pseudocode

```
1 FUNCTION do_something(x, y)
2     IF x == 1 THEN
3         RETURN y
4     ELSE
5         RETURN do_something(x-1, x+y)
6     ENDIF
7 ENDFUNCTION
```

Trace the subroutine to determine what the final return value will be when the following call is made:

`do_something(5, 2)`

Quiz:

STEM SMART Computer Science Week 14

All teaching materials on this site are available under a CC BY-NC-SA 4.0 license, except where otherwise stated.

